Practical Session 4: Mastering Loops in Java

Student-Centered, Self-Paced Lab

➢ Learning Outcomes Covered

By the end of this practical session, you will be able to:

1. **Analyze** the loop efficiency of different algorithms (linear vs nested loops) by counting iterations.
2. **Choose** the appropriate loop type (for, while, do-while) for different repetition scenarios.
3. **Debug** infinite loops by tracing variable updates and loop conditions.
4. **Create** a program that uses nested loops to generate complex patterns (e.g., triangle, pyramid, multiplication table).

Time Required: 120–150 minutes
Instructor Intervention: None – you will figure it out yourself!

➢ Before You Begin

Ensure you have completed **Week 1, 2, and 3 practical sessions**. You will need:

- VS Code open
- Your java-course folder with week3 subfolder
- Git configured and connected to GitHub

➢ Your Roadmap



TASK 1: The while Loop – Guessing Game

Objective

Use a while loop to create a number guessing game where the loop continues until the user guesses correctly.

Step 1.1: Create a New Folder

1. Open **VS Code**.
2. Open your java-course folder.
3. Right-click the java-course folder → **New Folder**.
4. Name it: week4
5. Press Enter.

Step 1.2: Create the File

1. Right-click the week4 folder → **New File**.

2. Name the file: GuessingGame.java
3. Press Enter.

Step 1.3: Write the Code

Copy and paste this code into GuessingGame.java:

java

```
// This program demonstrates the while loop using a number guessing game.
// The loop continues until the user guesses the correct number.

// Import the Scanner class for user input.
import java.util.Scanner;

public class GuessingGame {
    public static void main(String[] args) {

        // Create a Scanner object to read input from the keyboard.
        Scanner scanner = new Scanner(System.in);

        // Generate a random number between 1 and 100.
        // Math.random() returns a double between 0.0 and 1.0.
        // Multiplying by 100 gives 0.0 to 99.999...
        // Casting to int gives 0 to 99.
        // Adding 1 gives 1 to 100.
        int secretNumber = (int) (Math.random() * 100) + 1;

        // Declare a variable to store the user's guess.
        // Initialize to 0 so the loop condition is initially true.
```

```
int guess = 0;

// Declare a counter to track the number of attempts.
int attempts = 0;

// Display welcome message.
System.out.println("=======================================");
System.out.println("     NUMBER GUESSING GAME");
System.out.println("=======================================");
System.out.println("I'm thinking of a number between 1 and 100.");
System.out.println("Can you guess it?\n");

// ============================================================
// WHILE LOOP: Continue until the guess matches the secret number.
// ============================================================
// The loop condition is checked BEFORE each iteration.
// As long as guess != secretNumber, the loop body executes.
// ============================================================

while (guess != secretNumber) {

   // Increment the attempt counter.
   attempts++;

   // Prompt the user to enter a guess.
   System.out.print("Attempt " + attempts + " - Enter your guess: ");
```

```
    // Read the user's guess.
    guess = scanner.nextInt();

    // ============================================================
    // SELECTION CONTROL: Provide feedback based on the guess.
    // ============================================================

    // Check if the guess is too low.
    if (guess < secretNumber) {
        System.out.println("Too low! Try again.\n");
    }
    // Check if the guess is too high.
    else if (guess > secretNumber) {
        System.out.println("Too high! Try again.\n");
    }
    // If neither too low nor too high, the guess must be correct.
    else {
        System.out.println("\n 🎉 Congratulations! You guessed it!");
        System.out.println("The number was: " + secretNumber);
        System.out.println("You took " + attempts + " attempt(s).");
    }
}

// Close the scanner to free resources.
scanner.close();
```

```
    }
}
```

Step 1.4: Run the Program

1. Press Ctrl + S to save.
2. Click the **Run** button (▶️) at the top-right of VS Code.
3. Play the game! Try to guess the number.

Sample Interaction:

```
==========================================
    NUMBER GUESSING GAME
==========================================
I'm thinking of a number between 1 and 100.
Can you guess it?

Attempt 1 - Enter your guess: 50
Too low! Try again.

Attempt 2 - Enter your guess: 75
Too high! Try again.

Attempt 3 - Enter your guess: 62
Too low! Try again.

Attempt 4 - Enter your guess: 68
```

- ➢ Congratulations! You guessed it!

```
The number was: 68
You took 4 attempt(s).
```

Step 1.5: Answer These Questions

1. How many times does the loop execute if the user guesses correctly on the first try?
2. What happens if the user never guesses correctly?
3. Why is guess initialized to 0 before the loop?
4. What would happen if we removed attempts++ from inside the loop?

✅ **Task 1 Complete**

- □ I created GuessingGame.java
- □ I ran the program successfully
- □ I answered the questions

TASK 2: The do-while Loop – Menu System

Objective

Use a do-while loop to create a menu-driven program that repeats until the user chooses to exit.

Step 2.1: Create the File

Create a new file called MenuSystem.java in the week4 folder.

Step 2.2: Write the Code

java

```
// This program demonstrates the do-while loop using a menu system.
// The menu is displayed at least once, then repeats until the user exits.


// Import the Scanner class for user input.
import java.util.Scanner;


public class MenuSystem {
    public static void main(String[] args) {
```

```
// Create a Scanner object to read input from the keyboard.
Scanner scanner = new Scanner(System.in);

// Declare a variable to store the user's menu choice.
// Initialize to 0 so we can enter the loop.
int choice = 0;

// ============================================================
// DO-WHILE LOOP: Display menu at least once, repeat until exit.
// ============================================================
// The loop body executes FIRST, then the condition is checked.
// This guarantees the menu appears at least once.
// ============================================================

do {
    // Display the menu options.
    System.out.println("\n========================================");
    System.out.println("     STUDENT MANAGEMENT SYSTEM");
    System.out.println("========================================");
    System.out.println("1. Add Student");
    System.out.println("2. View All Students");
    System.out.println("3. Search Student");
    System.out.println("4. Exit");
    System.out.println("========================================");
    System.out.print("Enter your choice (1-4): ");
```

```
// Read the user's choice.
choice = scanner.nextInt();
scanner.nextLine(); // Consume the newline character.

// ============================================================
// SWITCH STATEMENT: Handle the user's choice.
// ============================================================

switch (choice) {
    case 1:
        // Option 1: Add Student
        System.out.print("Enter student name: ");
        String name = scanner.nextLine();
        System.out.println("✓ Student '" + name + "' added successfully.");
        break;

    case 2:
        // Option 2: View All Students
        System.out.println("Displaying all students...");
        System.out.println("(This is a simulation - no data stored yet.)");
        break;

    case 3:
        // Option 3: Search Student
        System.out.print("Enter student name to search: ");
```

```
            String searchName = scanner.nextLine();
            System.out.println("Searching for '" + searchName + "'...");
            System.out.println("(This is a simulation - no data stored yet.)");
            break;

        case 4:
            // Option 4: Exit
            System.out.println("\nThank you for using the Student Management System.");
            System.out.println("Goodbye!");
            break;

        default:
            // Invalid choice
            System.out.println("✗ Invalid choice. Please enter 1, 2, 3, or 4.");
            break;
      }

    } while (choice != 4); // Continue until user chooses 4 (Exit).

    // Close the scanner to free resources.
    scanner.close();
  }
}
```

Step 2.3: Run the Program

Save and run. Try each option. Select option 4 to exit.

Sample Interaction:

```
========================================
    STUDENT MANAGEMENT SYSTEM
========================================
1. Add Student
2. View All Students
3. Search Student
4. Exit
========================================
Enter your choice (1-4): 1
Enter student name: Alice Wanjiku
✓ Student 'Alice Wanjiku' added successfully.

========================================
    STUDENT MANAGEMENT SYSTEM
========================================
...
Enter your choice (1-4): 4

Thank you for using the Student Management System.
Goodbye!
```

Step 2.4: Answer These Questions

1. Why is do-while better than while for a menu system?
2. How many times does the menu display if the user enters 4 on the first try?
3. What happens if the user enters an invalid number like 5?
4. What would happen if we used a while loop instead of do-while?

✅ **Task 2 Complete**

- □ I created MenuSystem.java
- □ I ran the program successfully
- □ I answered the questions

TASK 3: The for Loop – Counter-Controlled Repetition

Objective

Use a for loop to perform calculations a known number of times.

Step 3.1: Create the File

Create a new file called ForLoopDemo.java in the week4 folder.

Step 3.2: Write the Code

java

```
// This program demonstrates the for loop with various examples.
// The for loop is ideal when the number of iterations is known.


public class ForLoopDemo {
    public static void main(String[] args) {


        // ============================================================
        // EXAMPLE 1: Basic for loop - print numbers 1 to 5
        // ============================================================

        System.out.println("=== EXAMPLE 1: Numbers 1 to 5 ===");


        // for (initialization; condition; update)
        // initialization: int i = 1 (runs once)
        // condition: i <= 5 (checked before each iteration)
```

```
// update: i++ (runs after each iteration)
for (int i = 1; i <= 5; i++) {
   System.out.println("Number: " + i);
}

// ============================================================
// EXAMPLE 2: Sum of numbers 1 to 100
// ============================================================

System.out.println("\n=== EXAMPLE 2: Sum of 1 to 100 ===");

// Declare a variable to hold the sum.
int sum = 0;

// Loop from 1 to 100.
for (int i = 1; i <= 100; i++) {
   // Add the current number to the sum.
   sum += i;
}

// Print the total sum.
System.out.println("Sum of numbers 1 to 100 = " + sum);

// ============================================================
// EXAMPLE 3: Countdown from 10 to 1
// ============================================================
```

```
System.out.println("\n=== EXAMPLE 3: Countdown ===");

// Loop from 10 down to 1 (decrement).
for (int i = 10; i >= 1; i--) {
   System.out.print(i + " ");
}
System.out.println("Blast off! 🚀 ");

// ============================================================
// EXAMPLE 4: Print even numbers from 2 to 20
// ============================================================

System.out.println("\n=== EXAMPLE 4: Even Numbers 2 to 20 ===");

// Loop from 2 to 20, incrementing by 2.
for (int i = 2; i <= 20; i += 2) {
   System.out.print(i + " ");
}
System.out.println();

// ============================================================
// EXAMPLE 5: Multiplication table for 7
// ============================================================

System.out.println("\n=== EXAMPLE 5: Multiplication Table for 7 ===");
```

```
        // Loop from 1 to 10.
        for (int i = 1; i <= 10; i++) {
            // Print the multiplication result.
            System.out.println("7 x " + i + " = " + (7 * i));
        }

        // ============================================================
        // EXAMPLE 6: Enhanced for loop (for-each) with an array
        // ============================================================

        System.out.println("\n=== EXAMPLE 6: Enhanced for Loop ===");

        // Declare and initialize an array of names.
        String[] students = {"Alice", "Bob", "Charlie", "Diana"};

        // Enhanced for loop: for (Type variable : collection)
        // Reads as: "for each student in students"
        for (String student : students) {
            System.out.println("Student: " + student);
        }
    }
}
```

Step 3.3: Run the Program

Save and run. Observe each example's output.

Expected Output:

text

=== EXAMPLE 1: Numbers 1 to 5 ===

Number: 1

Number: 2

Number: 3

Number: 4

Number: 5

=== EXAMPLE 2: Sum of 1 to 100 ===

Sum of numbers 1 to 100 = 5050

=== EXAMPLE 3: Countdown ===

10 9 8 7 6 5 4 3 2 1 Blast off!

=== EXAMPLE 4: Even Numbers 2 to 20 ===

2 4 6 8 10 12 14 16 18 20

=== EXAMPLE 5: Multiplication Table for 7 ===

7 x 1 = 7

7 x 2 = 14

...

7 x 10 = 70

=== EXAMPLE 6: Enhanced for Loop ===

Student: Alice

Student: Bob

Student: Charlie

Student: Diana

Step 3.4: Answer These Questions

1. How many times does the loop in Example 1 execute?
2. What is the final value of sum in Example 2?
3. What is the difference between i++ and i--?
4. What does i += 2 mean in Example 4?
5. When would you use an enhanced for loop instead of a traditional for loop?

✅ **Task 3 Complete**

- □ I created ForLoopDemo.java
- □ I ran the program successfully
- □ I answered the questions

TASK 4: Nested Loops – Pattern Generator

Objective

Use nested loops to generate complex patterns.

Step 4.1: Create the File

Create a new file called PatternGenerator.java in the week4 folder.

Step 4.2: Write the Code

java

```
// This program demonstrates nested loops by generating various patterns.


// Import Scanner for user input.
import java.util.Scanner;


public class PatternGenerator {
```

```
public static void main(String[] args) {

    // Create a Scanner object.
    Scanner scanner = new Scanner(System.in);

    // Ask the user for the number of rows.
    System.out.print("Enter the number of rows: ");
    int rows = scanner.nextInt();

    // ============================================================
    // PATTERN 1: Right Triangle (Increasing Stars)
    // ============================================================

    System.out.println("\n=== PATTERN 1: Right Triangle ===");

    // Outer loop controls the number of rows.
    for (int i = 1; i <= rows; i++) {

        // Inner loop prints stars for the current row.
        // For row i, it prints i stars.
        for (int j = 1; j <= i; j++) {
            System.out.print("*");
        }

        // Move to the next line after printing stars for this row.
        System.out.println();
```

```
}

// ============================================================
// PATTERN 2: Inverted Right Triangle (Decreasing Stars)
// ============================================================

System.out.println("\n=== PATTERN 2: Inverted Right Triangle ===");

// Outer loop counts down from rows to 1.
for (int i = rows; i >= 1; i--) {

   // Inner loop prints stars for the current row.
   // For row i, it prints i stars.
   for (int j = 1; j <= i; j++) {
      System.out.print("*");
   }

   // Move to the next line.
   System.out.println();
}

// ============================================================
// PATTERN 3: Pyramid (Centered Stars)
// ============================================================

System.out.println("\n=== PATTERN 3: Pyramid ===");
```

```
// Outer loop controls the number of rows.
for (int i = 1; i <= rows; i++) {

    // First inner loop: print spaces for alignment.
    // Number of spaces = rows - i
    for (int j = 1; j <= rows - i; j++) {
        System.out.print(" ");
    }

    // Second inner loop: print stars.
    // Number of stars = 2 * i - 1
    for (int k = 1; k <= 2 * i - 1; k++) {
        System.out.print("*");
    }

    // Move to the next line.
    System.out.println();
}

// ============================================================
// PATTERN 4: Number Triangle
// ============================================================

System.out.println("\n=== PATTERN 4: Number Triangle ===");
```

```
// Outer loop controls the number of rows.
for (int i = 1; i <= rows; i++) {

    // Inner loop prints numbers from 1 to i.
    for (int j = 1; j <= i; j++) {
        // Print the current number followed by a space.
        System.out.print(j + " ");
    }

    // Move to the next line.
    System.out.println();
}

// ============================================================
// PATTERN 5: Multiplication Table
// ============================================================

System.out.println("\n=== PATTERN 5: Multiplication Table ===");

// Print header row.
System.out.print("    ");
for (int i = 1; i <= rows; i++) {
    // Print column headers with formatting.
    System.out.printf("%4d", i);
}
System.out.println();
```

```
    // Print separator line.
    System.out.print("    ");
    for (int i = 1; i <= rows; i++) {
        System.out.print("----");
    }
    System.out.println();

    // Outer loop for each row.
    for (int i = 1; i <= rows; i++) {
        // Print row header.
        System.out.printf("%3d|", i);

        // Inner loop for each column.
        for (int j = 1; j <= rows; j++) {
            // Print the product with formatting.
            System.out.printf("%4d", i * j);
        }

        // Move to the next line.
        System.out.println();
    }

    // Close the scanner.
    scanner.close();
}
```

}

Step 4.3: Run the Program

Save and run. Enter 5 when prompted.

Expected Output:

text

Enter the number of rows: 5

=== PATTERN 1: Right Triangle ===

*

**

***

****

*****

=== PATTERN 2: Inverted Right Triangle ===

*****

****

***

**

*

=== PATTERN 3: Pyramid ===

    *

   ***

  *****

 ****

****

=== PATTERN 4: Number Triangle ===

1

1 2

1 2 3

1 2 3 4

1 2 3 4 5

=== PATTERN 5: Multiplication Table ===

```
    1   2   3   4   5
  ---------------------
 1|   1   2   3   4   5
 2|   2   4   6   8  10
 3|   3   6   9  12  15
 4|   4   8  12  16  20
 5|   5  10  15  20  25
```

Step 4.4: Experiment

Change the value of rows to:

- 3 → What happens?
- 7 → What happens?

Step 4.5: Answer These Questions

1. How many times does the inner loop execute in Pattern 1 when rows = 5?
2. What is the total number of stars printed in Pattern 1 when rows = 5?
3. In Pattern 3, how many spaces are printed in the first row?
4. In Pattern 3, how many stars are printed in the last row?

5. How would you modify Pattern 1 to print numbers instead of stars?

✅ **Task 4 Complete**

- □ I created PatternGenerator.java
- □ I ran the program successfully
- □ I experimented with different row values
- □ I answered the questions

TASK 5: Loop Efficiency Analysis

Objective

Analyze the efficiency of different loop algorithms by counting iterations.

Step 5.1: Create the File

Create a new file called LoopEfficiency.java in the week4 folder.

Step 5.2: Write the Code

java

```
// This program analyzes the efficiency of different loop algorithms.
// It counts the number of iterations for each algorithm.

public class LoopEfficiency {
    public static void main(String[] args) {

        // ============================================================
        // ALGORITHM 1: Linear Loop - O(n)
        // ============================================================

        System.out.println("=== ALGORITHM 1: Linear Loop ===");
```

```
// Declare a counter to track iterations.
int count1 = 0;

// Loop n times.
int n = 10;
for (int i = 0; i < n; i++) {
   // Increment the counter.
   count1++;
}

System.out.println("n = " + n);
System.out.println("Iterations: " + count1);
System.out.println("Time Complexity: O(n)");

// ============================================================
// ALGORITHM 2: Nested Loop - O(n^2)
// ============================================================

System.out.println("\n=== ALGORITHM 2: Nested Loop (Square) ===");

// Declare a counter to track iterations.
int count2 = 0;

// Outer loop runs n times.
for (int i = 0; i < n; i++) {
```

```
    // Inner loop runs n times for each outer iteration.
    for (int j = 0; j < n; j++) {
        // Increment the counter.
        count2++;
    }
}

System.out.println("n = " + n);
System.out.println("Iterations: " + count2);
System.out.println("Time Complexity: O(n^2)");
System.out.println("Formula: n * n = " + (n * n));

// ============================================================
// ALGORITHM 3: Triangular Loop - O(n^2/2)
// ============================================================

System.out.println("\n=== ALGORITHM 3: Triangular Loop ===");

// Declare a counter to track iterations.
int count3 = 0;

// Outer loop runs n times.
for (int i = 0; i < n; i++) {

    // Inner loop runs from i to n.
    for (int j = i; j < n; j++) {
```

```
        // Increment the counter.
        count3++;
    }
}

System.out.println("n = " + n);
System.out.println("Iterations: " + count3);
System.out.println("Formula: n * (n + 1) / 2 = " + (n * (n + 1) / 2));

// ============================================================
// ALGORITHM 4: Loop with Condition - O(n)
// ============================================================

System.out.println("\n=== ALGORITHM 4: Loop with Condition ===");

// Declare a counter to track iterations.
int count4 = 0;

// Loop n times, but only count even numbers.
for (int i = 0; i < n; i++) {
    // Check if i is even.
    if (i % 2 == 0) {
        // Increment the counter only for even numbers.
        count4++;
    }
}
```

```
        System.out.println("n = " + n);
        System.out.println("Even numbers counted: " + count4);
        System.out.println("Total iterations: " + n);
        System.out.println("Time Complexity: O(n)");

        // ============================================================
        // COMPARISON TABLE
        // ============================================================

        System.out.println("\n=== COMPARISON TABLE ===");
        System.out.println("Algorithm | Iterations | Complexity");
        System.out.println("----------|------------|------------");
        System.out.println("Linear    | " + count1 + "         | O(n)");
        System.out.println("Nested    | " + count2 + "        | O(n^2)");
        System.out.println("Triangular| " + count3 + "        | O(n^2/2)");
        System.out.println("Condition | " + count4 + "         | O(n)");
    }
}
```

Step 5.3: Run the Program

Save and run. Observe the iteration counts.

Expected Output:

text

```
=== ALGORITHM 1: Linear Loop ===
n = 10
Iterations: 10
```

Time Complexity: O(n)

=== ALGORITHM 2: Nested Loop (Square) ===

n = 10

Iterations: 100

Time Complexity: O(n^2)

Formula: n * n = 100

=== ALGORITHM 3: Triangular Loop ===

n = 10

Iterations: 55

Formula: n * (n + 1) / 2 = 55

=== ALGORITHM 4: Loop with Condition ===

n = 10

Even numbers counted: 5

Total iterations: 10

Time Complexity: O(n)

=== COMPARISON TABLE ===

Algorithm | Iterations | Complexity

----------|------------|------------

Linear | 10 | O(n)

Nested | 100 | O(n^2)

Triangular| 55 | O(n^2/2)

Condition | 5 | O(n)

Step 5.4: Experiment

Change n to 5 and 20. What happens to the iteration counts?

Step 5.5: Answer These Questions

1. Why does the nested loop (Algorithm 2) have 100 iterations when n=10?
2. Why does the triangular loop (Algorithm 3) have 55 iterations?
3. Which algorithm is most efficient? Why?
4. What would happen to the nested loop if n=100? How many iterations?
5. Why is it important to analyze loop efficiency?

✅ **Task 5 Complete**

- □ I created LoopEfficiency.java
- □ I ran the program successfully
- □ I experimented with different n values
- □ I answered the questions

TASK 6: Debugging Infinite Loops

Objective

Debug infinite loops by tracing variable updates and loop conditions.

Step 6.1: Examine the Buggy Code

Create a file called InfiniteLoopDebug.java in the week4 folder.

java

```
// This program contains infinite loops. Can you find and fix them?


public class InfiniteLoopDebug {
    public static void main(String[] args) {

        // ============================================================
```

```
// BUG 1: Missing increment
// ============================================================

System.out.println("=== BUG 1 ===");

int x = 1;

// The intention: print numbers 1 to 5.
// BUG: x is never incremented, so the condition x <= 5 is always true.
// FIX: Add x++ inside the loop.
while (x <= 5) {
   System.out.println("x = " + x);
   // Missing: x++;
}

// ============================================================
// BUG 2: Wrong direction
// ============================================================

System.out.println("\n=== BUG 2 ===");

// The intention: print numbers 5 down to 1.
// BUG: i starts at 5 and increments (i++), so i is never >= 1.
// FIX: Change i++ to i--.
for (int i = 5; i >= 1; i++) {
   System.out.println("i = " + i);
```

```
        }

        // ============================================================
        // BUG 3: Wrong condition
        // ============================================================

        System.out.println("\n=== BUG 3 ===");

        int y = 10;

        // The intention: countdown from 10 to 1.
        // BUG: Condition is y > 0, but y is decremented only after checking.
        // Actually, this loop works! Let's change the bug.

        // Real BUG: Condition is always true because y is not decremented.
        // FIX: Add y-- inside the loop.
        while (y > 0) {
            System.out.println("y = " + y);
            // Missing: y--;
        }
    }
}
```

Step 6.2: Identify the Bugs

Before running, analyze each bug:

Bug 1:

- **Problem:** x is never incremented, so x <= 5 is always true.

- **Fix:** Add x++; inside the loop.

Bug 2:

- **Problem:** i increments from 5, so i >= 1 remains true indefinitely.
- **Fix:** Change i++ to i--.

Bug 3:

- **Problem:** y is never decremented, so y > 0 is always true.
- **Fix:** Add y--; inside the loop.

Step 6.3: Create the Fixed Version

Create a new file called FixedLoops.java:

java

```
// This program contains the corrected versions of the infinite loops.


public class FixedLoops {
    public static void main(String[] args) {

        // ============================================================
        // FIXED BUG 1: Print numbers 1 to 5
        // ============================================================

        System.out.println("=== FIXED BUG 1 ===");

        int x = 1;

        // Loop while x is less than or equal to 5.
        while (x <= 5) {
            System.out.println("x = " + x);
```

```
    // FIX: Increment x so the condition eventually becomes false.
    x++;
}

// ============================================================
// FIXED BUG 2: Countdown from 5 to 1
// ============================================================

System.out.println("\n=== FIXED BUG 2 ===");

// FIX: Change i++ to i-- so i decreases toward 1.
for (int i = 5; i >= 1; i--) {
    System.out.println("i = " + i);
}

// ============================================================
// FIXED BUG 3: Countdown from 10 to 1
// ============================================================

System.out.println("\n=== FIXED BUG 3 ===");

int y = 10;

// Loop while y is greater than 0.
while (y > 0) {
    System.out.println("y = " + y);
```

```
        // FIX: Decrement y so the condition eventually becomes false.
        y--;
      }

      System.out.println("\nAll loops terminated successfully!");
   }
}
```

Step 6.4: Run the Fixed Program

Save and run. Verify all loops terminate correctly.

Expected Output:

text

```
=== FIXED BUG 1 ===
x = 1
x = 2
x = 3
x = 4
x = 5

=== FIXED BUG 2 ===
i = 5
i = 4
i = 3
i = 2
i = 1

=== FIXED BUG 3 ===
```

y = 10

y = 9

y = 8

y = 7

y = 6

y = 5

y = 4

y = 3

y = 2

y = 1

All loops terminated successfully!

Step 6.5: Answer These Questions

1. What is an infinite loop?
2. What are three common causes of infinite loops?
3. How can you identify an infinite loop?
4. How do you fix an infinite loop?
5. Why is it important to trace variable updates?

✅ **Task 6 Complete**

- □ I identified all three bugs
- □ I created FixedLoops.java
- □ I ran the fixed program successfully
- □ I answered the questions

TASK 7: Pus to GitHub

Step 7.1: Stage and Commit

1. Open the **Source Control** panel in VS Code.
2. Stage all new files in week4.
3. Commit with message: Week 4: Loops - while, do-while, for, nested loops, debugging
4. Push to GitHub.

Step 7.2: Verify on GitHub

Go to your repository and confirm all Week 4 files are there.

✅ **Task 7 Complete**

- □ I committed and pushed all files
- □ I verified on GitHub

➢ **Final Self-Assessment Checklist**

- □ I can use a while loop for indefinite repetition
- □ I can use a do-while loop for menu-driven programs
- □ I can use a for loop for counter-controlled repetition
- □ I can use an enhanced for loop to iterate through arrays
- □ I can use nested loops to generate patterns
- □ I can analyze loop efficiency by counting iterations
- □ I can identify and fix infinite loops
- □ I pushed all Week 4 code to GitHub

➢ **Troubleshooting Guide**

Problem	Solution
Infinite loop	Check that loop variable is updated and moves toward termination

Problem	Solution
InputMismatchException	You entered text instead of a number
Loop doesn't execute	Check that condition is initially true
nextLine() skipped after nextInt()	Add an extra scanner.nextLine() to consume newline
Nested loop too slow	Reduce n or optimize algorithm
Pattern not aligned	Count spaces and stars carefully

- **Reflection Questions**

1. Which loop type did you find most useful? Why?
2. What was the most challenging part of this practical session?
3. What error message did you encounter, and how did you solve it?
4. In your own words, when would you use while vs do-while vs for?
5. How will loops help you build real-world applications?

- **What You've Achieved**

You have just:

- Mastered the while **loop** with a guessing game
- Built a **menu system** with the do-while **loop**
- Used the for **loop** for various calculations
- Created **complex patterns** with **nested loops**
- **Analyzed** loop efficiency

- **Debugged** infinite loops

You can now make your programs repeat tasks efficiently!

➢ Next Steps

In Week 5, you will:

1. Learn about **arrays** and **ArrayLists**
2. Store and manage collections of data
3. Build a **student records system** and **shopping cart**
4. Continue pushing your code to GitHub

➢ Additional Resources

Topic	Resource
while loop	https://docs.oracle.com/javase/tutorial/java/nutsandbolts/while.html
for loop	https://docs.oracle.com/javase/tutorial/java/nutsandbolts/for.html
Nested loops	https://docs.oracle.com/javase/tutorial/java/nutsandbolts/for.html
Loop efficiency	https://www.geeksforgeeks.org/analysis-algorithms-big-o-analysis/

END OF PRACTICAL SESSION 4

"The best way to learn is by doing." – Anonymous

Instructor Note: You should complete this lab independently. I encourage you to experiment with different loop values and patterns. The goal is to build intuition for when to use each loop type and how to debug common loop errors.